Session 04 – Recurrent Neural Networks and Transformers

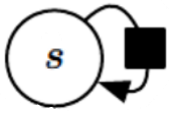
Dr Ivan Olier
I.Olier@ljmu.ac.uk

1

Dynamical systems

- Classical form of a dynamical system:

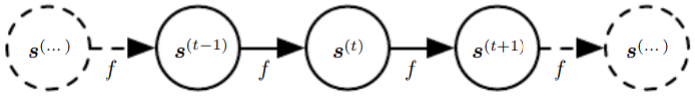
$$s^{(t)} = f(s^{(t-1)}; \theta)$$



- It is recurrent because the definition of the state of the system, s, at time t refers back to the same definition at time $t-1$.
- For a finite number of time steps τ, the above equation can be unfolded by applying the definition $\tau - 1$ times.
- For example, if we unfold it for $\tau = 3$ timesteps, we obtain:

$$\begin{aligned} s^{(3)} &= f(s^{(2)}; \theta) \\ &= f(f(s^{(1)}; \theta); \theta) \end{aligned}$$

- Which can be represer

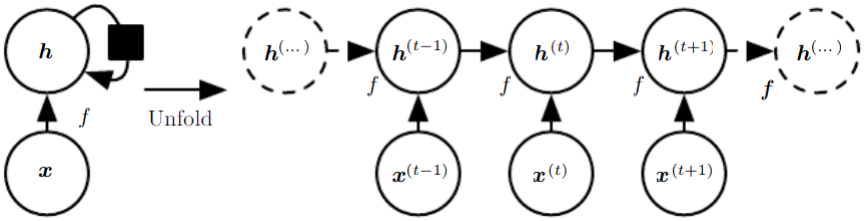


2

Dynamical systems driven by external signals

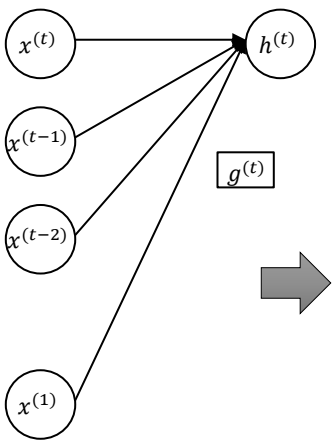
- A dynamical system can also be driven by an external signal.
- The state of the system, $\boldsymbol{h}$, at time t is defined by the state at time $t-1$ and the external signal $\boldsymbol{x}$ at time t.

$$\boldsymbol{h}^{(t)} = f(\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}; \boldsymbol{\theta})$$

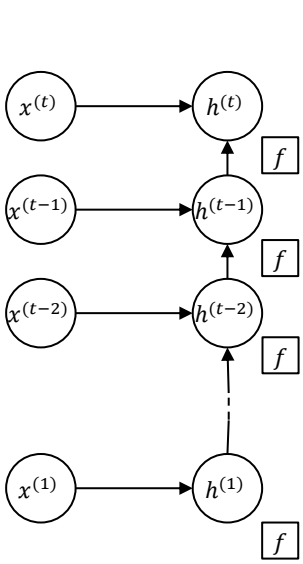


3

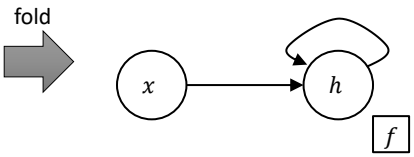
Parameter sharing



$$\boldsymbol{h}^{(t)} = g(\boldsymbol{x}^{(t)}, \boldsymbol{x}^{(t-1)}, \boldsymbol{x}^{(t-2)}, \ldots, \boldsymbol{x}^{(1)})$$



$$\boldsymbol{h}^{(t)} = f(\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}; \boldsymbol{\theta})$$

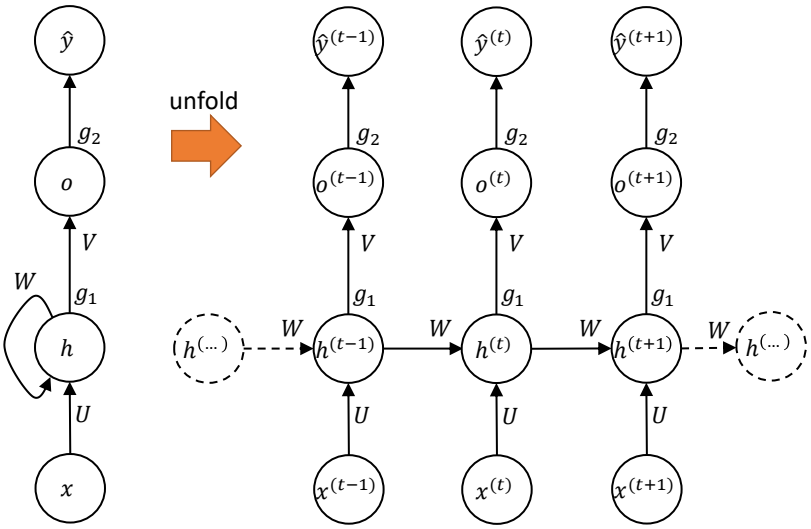


- Regardless of the sequence length, the model always has the same input size, because it is specified in terms of transition from one state to another state, rather than specified in terms of a variable-length history of states.
- It is possible to use the same transition function f with the same parameters at every time step (parameter sharing).

4

From dynamical systems to recurrent neural networks

Before dynamical system can be seen as a neural network:



$$\boldsymbol{a}^{(t)} = \boldsymbol{b} + \boldsymbol{W}\boldsymbol{h}^{(t-1)} + \boldsymbol{U}\boldsymbol{x}^{(t)}$$
$$\boldsymbol{h}^{(t)} = g_1\left(\boldsymbol{a}^{(t)}\right)$$
$$\boldsymbol{o}^{(t)} = \boldsymbol{c} + \boldsymbol{V}\boldsymbol{h}^{(t)}$$
$$\hat{\boldsymbol{y}}^{(t)} = g_2\left(\boldsymbol{o}^{(t)}\right)$$

Where:
- $\boldsymbol{W}$, $\boldsymbol{U}$ and $\boldsymbol{V}$ are weight matrices
- $\boldsymbol{b}$ and $\boldsymbol{c}$ are bias vectors
- g_1 and g_2 are activation functions

Recurrent Neural Networks (RNN)

- RNNs are, basically, dynamical systems.
- In general, they are a family of neural networks for processing sequential data.
- Much as a CNN is a neural network that is specialised for processing a grid of values X such as an image, a RNN is a neural network that is specialised for processing a sequence of values $x^{(1)}, \dots, x^{(\tau)}$.
- Just as convolutional networks can readily scale to images with large width and height, and some convolutional networks can process images of variable size, recurrent networks can scale to much longer sequences than would be practical for networks without sequence-based specialisation.
- Most recurrent networks can also process sequences of variable length.

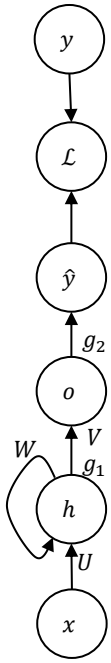
Training RNNs

- An unfolded RNN is basically a feedforward neural network with shared parameters (weights, biases, activation functions).
- Error back-propagation can still be used to find optimal parameter values.
- If y is the desired (true) output, $\hat{y}$, the predicted output, and $\mathcal{L}$ is the loss function (error function is just a particular case):

$$\mathcal{L}(y, \hat{y}) = \sum_{t=1}^{T} \mathcal{L}\left(y^{(t)}, \hat{y}^{(t)}\right)$$

- Then, error or loss backpropagation is done at each point in time. At timestep T, the derivative of the loss $\mathcal{L}$ with respect to the weights is expressed as follows:

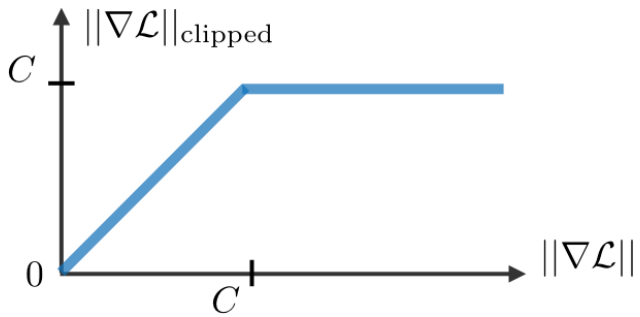
$$\frac{\partial \mathcal{L}^{(T)}}{\partial W} = \sum_{t=1}^{T} \left.\frac{\partial \mathcal{L}^{(T)}}{\partial W}\right|_{(t)}$$



The challenge of long-term dependencies

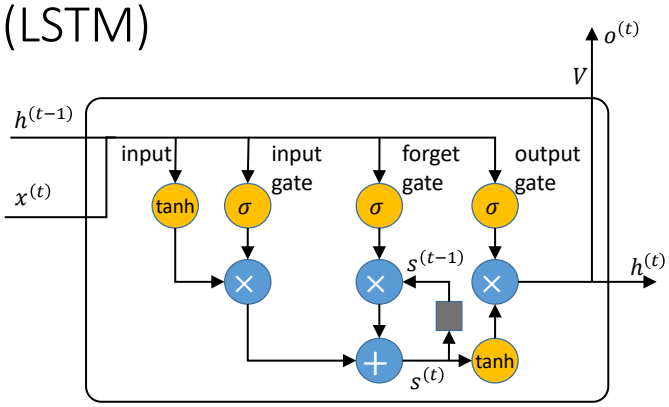
- Learning long-term dependencies in recurrent neural networks is a mathematical challenge.
- The basic problem is that gradients propagated over many stages tend to either vanish (most of the time) or explode (rarely, but with much damage to the optimisation).
- This happens because of multiplicative gradient that can be exponentially decreasing/increasing with respect to the number of layers.

- One solution – gradient clipping. As its name suggests, the technique caps the maximum value for the gradient.



Long short-term memory (LSTM)

- **LSTMs** are RNNs that have the ability of learning long-term dependencies.
- They efficiently control the risk of gradient vanishing/exploding by implementing:
  - A self-loop to accumulate information over a long duration
  - An output gate to control whether the message is passed on to the next cell.
  - A forget gate to control when to forget a state.
  - An input gate to control how past information is relevant to the cell.
- LSTMs are the most popular RNNs.
- They have been found extremely successful in many applications.



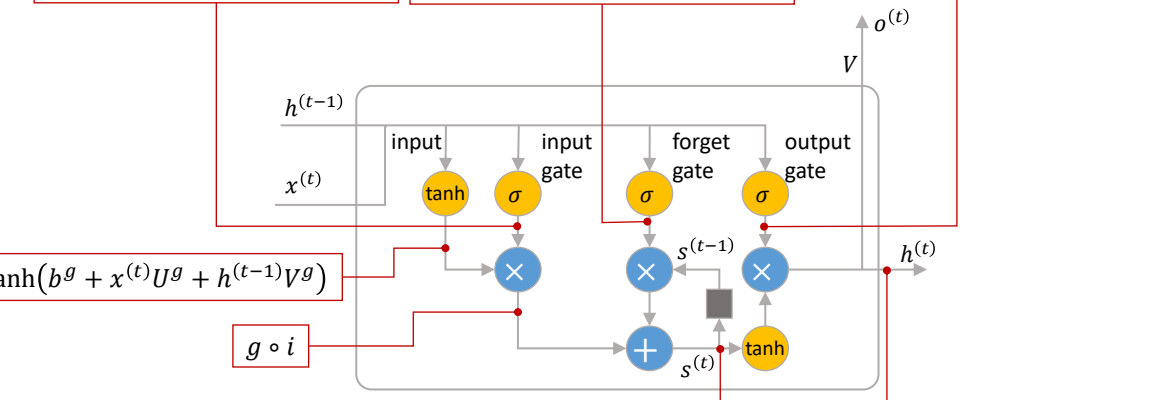
9

Long short-term memory (LSTM)

$o = \sigma\left(b^o + x^{(t)}U^o + h^{(t-1)}V^o\right)$

$i = \sigma\left(b^i + x^{(t)}U^i + h^{(t-1)}V^i\right)$

$f = \sigma\left(b^f + x^{(t)}U^f + h^{(t-1)}V^f\right)$



$g = \tanh\left(b^g + x^{(t)}U^g + h^{(t-1)}V^g\right)$

$g \circ i$

$h^{(t)} = \tanh\left(s^{(t)}\right) \circ o$

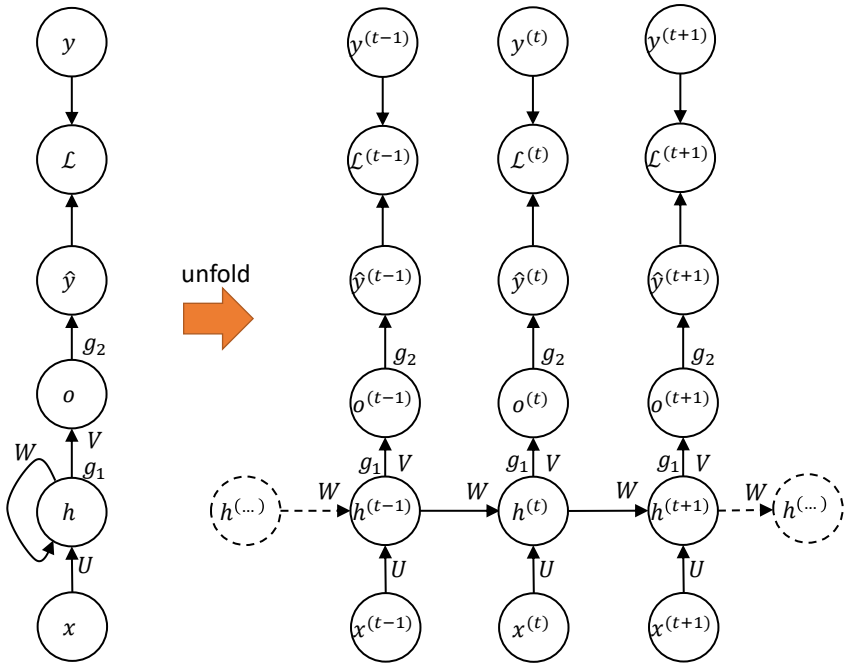
$s^{(t)} = s^{(t-1)} \circ f + g \circ i$

Operator '$\circ$' corresponds to element-wise multiplication.

10

Types of RNNs

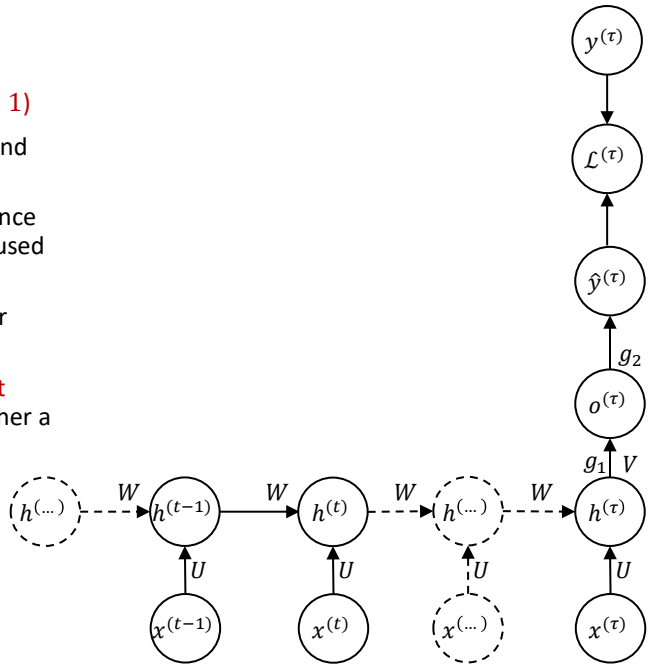
- RNNs are actually a family of ANNs.
- Similarly to the feedforward NNs, neurons or units can be connected in different forms.
- In the end, it will depend on the application or whether we want to speed up the training process or whether we want to exploit a particular property.
- So, we have seen one type so far.
- This type of RNN is called many-to-many ($T_x = T_y$), it has as many inputs as outputs.



11

Types of RNNs

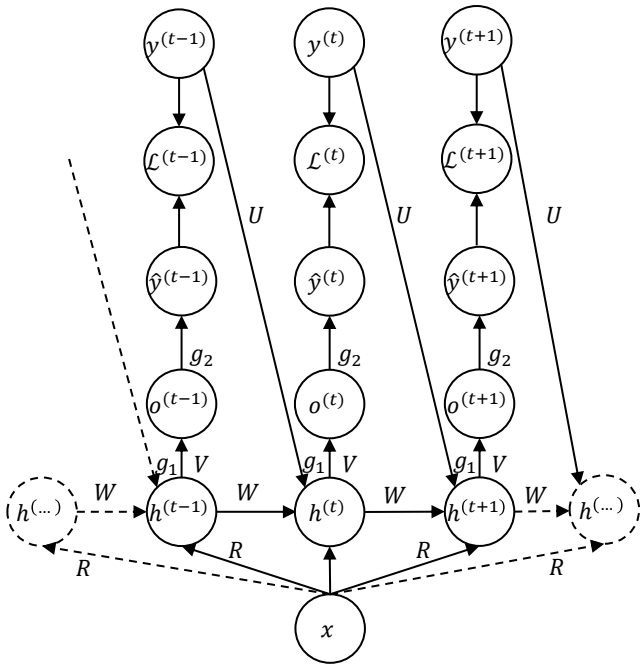
- This is a many-to-one RNN ($T_x > 1, T_y = 1$)
- The network takes a sequence as input and delivers a single output.
- It could be used to summarise the sequence and produce a fixed-size representation used as input for further processing.
- Or it can be directly used as a classifier or regressor.
- For instance, it can be used for sentiment analysis, where we want to predict whether a text fragment is positive or negative.



12

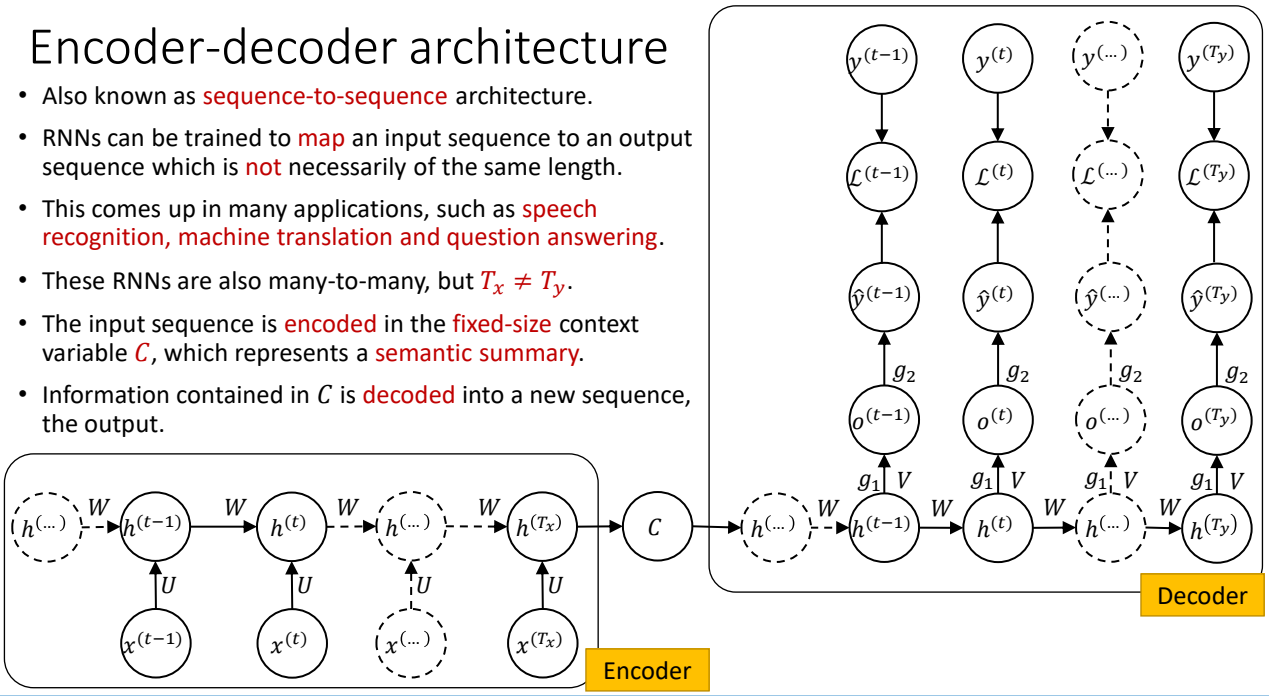
Types of RNNs

- An example of an one-to-many RNN ($T_x = 1$, $T_y > 1$).
- This RNN maps a fixed-length vector $\boldsymbol{x}$ into a distribution over sequences $\boldsymbol{Y}$.
- This RNN is appropriate for tasks such as image captioning, where a single image is used as input to a model that then produces a sequence of words describing the image.
- It has also been used for music generation.



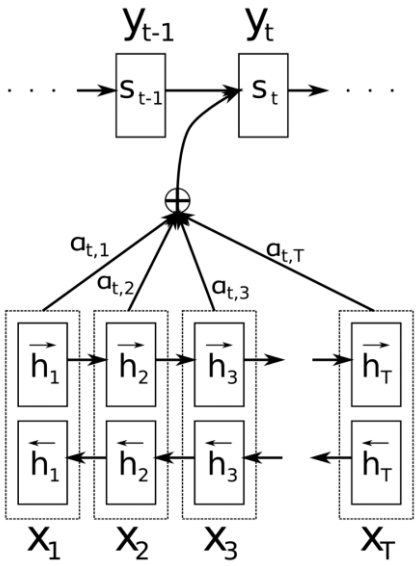
Encoder-decoder architecture

- Also known as sequence-to-sequence architecture.
- RNNs can be trained to map an input sequence to an output sequence which is not necessarily of the same length.
- This comes up in many applications, such as speech recognition, machine translation and question answering.
- These RNNs are also many-to-many, but $T_x \neq T_y$.
- The input sequence is encoded in the fixed-size context variable C, which represents a semantic summary.
- Information contained in C is decoded into a new sequence, the output.



Attention Mechanism

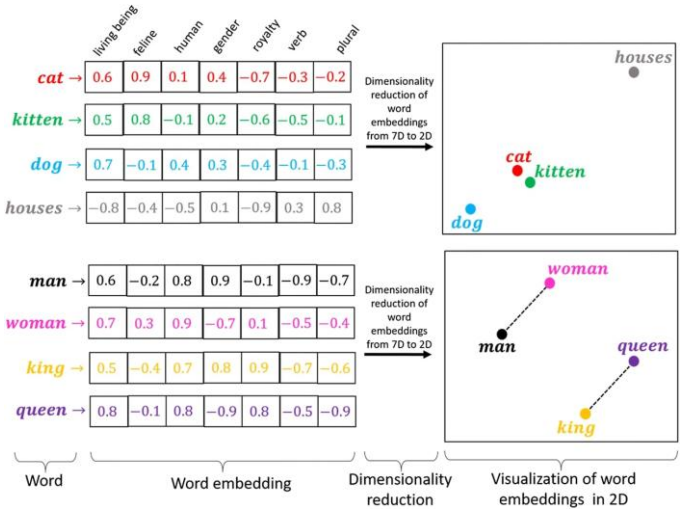
- The **Attention Mechanism** is a technique used in machine learning to help neural networks selectively focus on certain parts of an input sequence.
- It works by assigning **weights** to different parts of the input sequence based on their relevance to the current context or task.
- Attention Mechanism has become popular in **natural language processing (NLP)** tasks, as it can help models better understand the context of long sequences of text.
- First version: **encoder-decoder LSTMs**.
- However, LSTMs **cannot** remember long sentences or give more importance to some of the input words compared to others while translating the sentence.
- Bahdanau et al (2016) *Neural Machine Translation by Jointly Learning to Align and Translate*. https://arxiv.org/abs/1409.0473



15

Text encoding

- **Tokenisation**:
  - Splits text into smaller units (tokens).
  - Tokens can be words, subwords, or characters.
  - Example:
    - Sentence: "AI models process text“
    - Tokens: ["AI", "models", "process", "text"]
- **Word embeddings**:
  - Converts tokens into numerical vectors.
  - Vectors capture semantic meaning and relationships.
  - Example:
    - "King" - "Man" + "Woman" ≈ "Queen“
  - Common methods: Word2Vec, GloVe, Transformer Embeddings



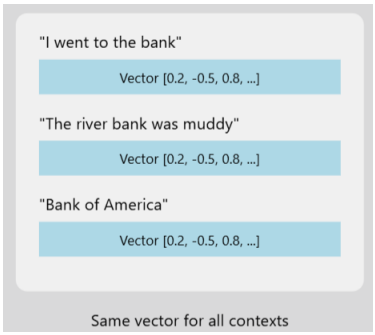
16

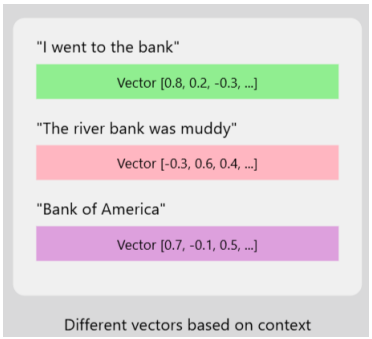
Contextual Embeddings

- Consider the word "**bank**" in the following sentences:
  - "Sitting on a bank next to a river"
  - "Thinking about my empty bank account"

- In static word embeddings, the vector representation for "**bank**" would be identical in both sentences, despite the word having different meanings in each context.

- Contextual embeddings generate vector representations that adapt based on the context in which a word appears.
- These embeddings are dynamic and can capture the nuanced meanings of words in different situations.
- In the example above, the vector for "bank" in the first sentence would differ from that in the second sentence, reflecting the different semantic contexts.





17

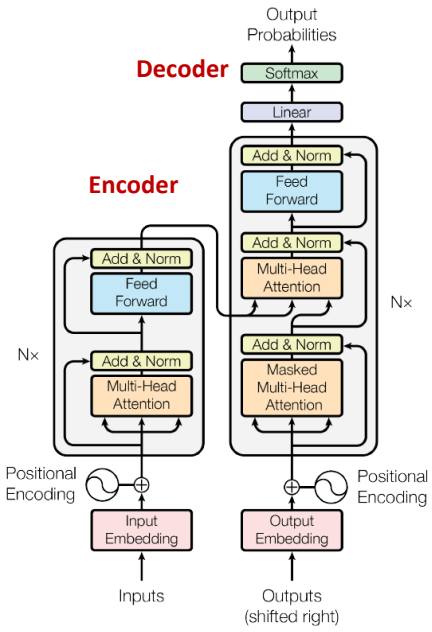
Transformers

- They use an **attention mechanism** to focus on different parts of the input sequence when processing it, making them particularly effective for tasks that involve **long sequences** of text.

- Follows an Encoder-Decoder architecture.

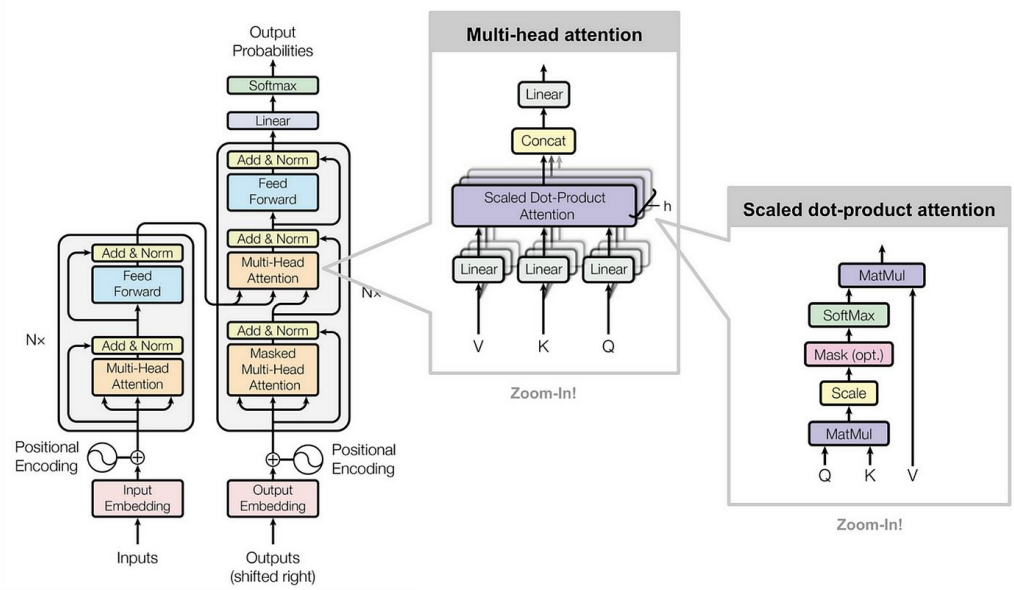
- Transformers have been used for a variety of NLP tasks, including **language modeling**, **machine translation**, and **sentiment analysis**, and have also been adapted for other applications such as *image recognition* and *speech recognition*.

- Transformers were introduced in the 2017 paper "Attention Is All You Need" by Vaswani et al. and have since become a popular choice for NLP tasks.



18

Transformers

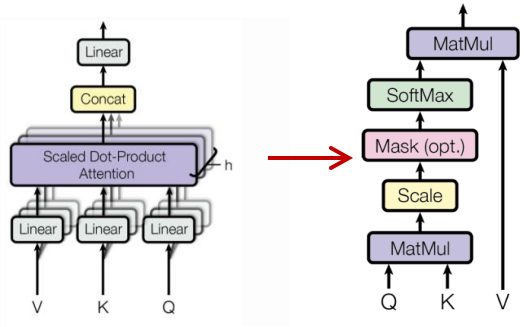


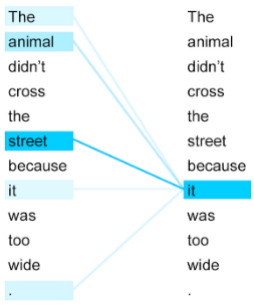
19

Self-attention

- Transformers implement a very generic and broad definition of Attention based on **key**, **query**, and **values:**
  - The **Key**, **Query**, and **Value** are three vectors that are used to compute the attention weights in the attention mechanism.
  - **Key**: It encapsulates information from the input sequence, serving as a basis for determining the importance of other elements in the sequence (*Catalog*).
  - **Query**: These representations identify elements within the sequence to which the model should allocate attention during computation (*Questions to pose*).
  - **Value**: These vectors hold the actual content extracted from the sequence, which is then weighted by the attention mechanism based on the corresponding queries and keys (*Relevance of words to the questions*).

- **Multi-headed attention**
  - It allows the model to attend to multiple positions in the input sequence simultaneously





20

Query, Key and Value

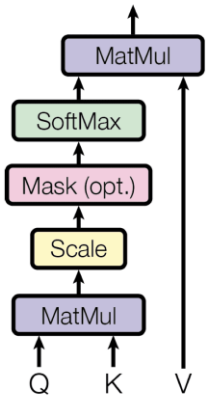
Input Matrix **Query Matrix** **Key Matrix** **Value Matrix**

$$\mathbf{X} = \begin{bmatrix} \leftarrow \mathbf{x}_1 \rightarrow \\ \leftarrow \mathbf{x}_2 \rightarrow \\ \vdots \\ \leftarrow \mathbf{x}_n \rightarrow \end{bmatrix} \quad \mathbf{Q} = \begin{bmatrix} \leftarrow \mathbf{q}_1 \rightarrow \\ \leftarrow \mathbf{q}_2 \rightarrow \\ \vdots \\ \leftarrow \mathbf{q}_n \rightarrow \end{bmatrix} = \mathbf{X}\,\mathbf{W}^Q \quad \mathbf{K} = \begin{bmatrix} \leftarrow \mathbf{k}_1 \rightarrow \\ \leftarrow \mathbf{k}_2 \rightarrow \\ \vdots \\ \leftarrow \mathbf{k}_n \rightarrow \end{bmatrix} = \mathbf{X}\,\mathbf{W}^K \quad \mathbf{V} = \begin{bmatrix} \leftarrow \mathbf{v}_1 \rightarrow \\ \leftarrow \mathbf{v}_2 \rightarrow \\ \vdots \\ \leftarrow \mathbf{v}_n \rightarrow \end{bmatrix} = \mathbf{X}\,\mathbf{W}^V$$

where:

- X is the **input embedding** matrix (or hidden representation from a previous layer).
- W^Q, W^K, and W^V are **learnable weight** matrices that project the input embeddings into the query, key, and value spaces, respectively.
- Q, K, and V are the resulting **query**, **key**, and **value** matrices.

Scaled Dot Product Attention

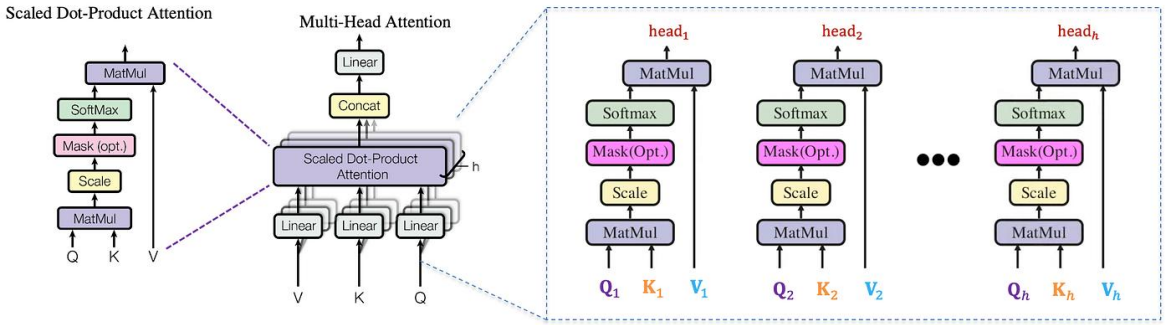


$$\mathbf{Z} = \begin{bmatrix} \leftarrow \mathbf{z}_1 \rightarrow \\ \leftarrow \mathbf{z}_2 \rightarrow \\ \vdots \\ \leftarrow \mathbf{z}_n \rightarrow \end{bmatrix} = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\mathrm{T}}}{\sqrt{d_k}}\right)\mathbf{V}$$

- Where QK^T are the **attention scores**, which are scaled ($\sqrt{d_k}$) and normalised using the *softmax* function.
- The dot product of Q and K measures the **similarity** between the query and the keys
  - If the similarity is high, the corresponding value is retrieved.
- In the context of self-attention, this process is applied to all tokens in the input sequence, allowing each token to attend to all other tokens.

Muti-Head Attention

$$\mathbf{O} = \text{Multihead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)\mathbf{W}^O$$

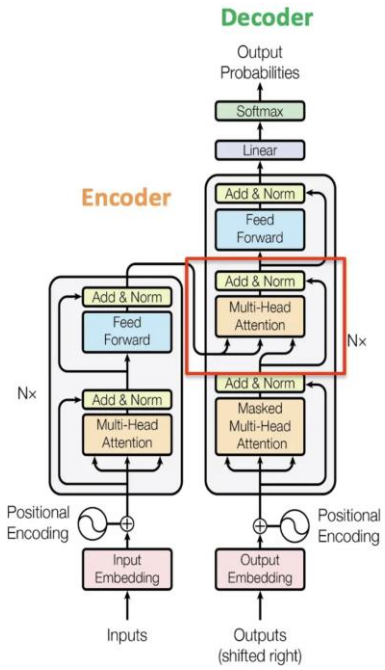


- **Multi-Head Attention** is an extension of self-attention, allowing the model to focus on different parts of the input sequence simultaneously.
- This is achieved by applying several attention mechanisms in parallel.
- The output of the multi-head attention is the weighted concatenation of the heads.

Cross-Attention

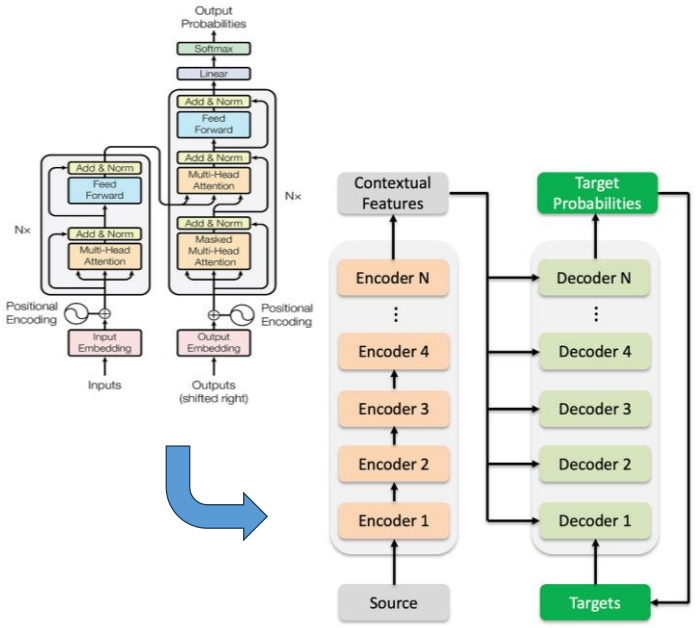
- **Cross-attention** connects encoder and decoder in sequence-to-sequence models.
  - Unlike self-attention, it links two different sequences.
- Decoder queries come from its hidden states; keys and values come from encoder outputs.
- Enables the decoder to focus on relevant encoder information for each output token.
- Cross-attention acts as the **bridge** between the encoder and decoder in a transformer model, **enabling** the model to learn different language conversion patterns between the two sequences, leading to more accurate and fluent translations.

$$\text{CrossAttention}(\mathbf{Q}_{\text{dec}}, \mathbf{K}_{\text{enc}}, \mathbf{V}_{\text{enc}}) = \text{Softmax}\left(\frac{\mathbf{Q}_{\text{dec}}\mathbf{K}_{\text{enc}}^{\mathbf{T}}}{\sqrt{d_k}}\right)\mathbf{V}_{\text{enc}}$$



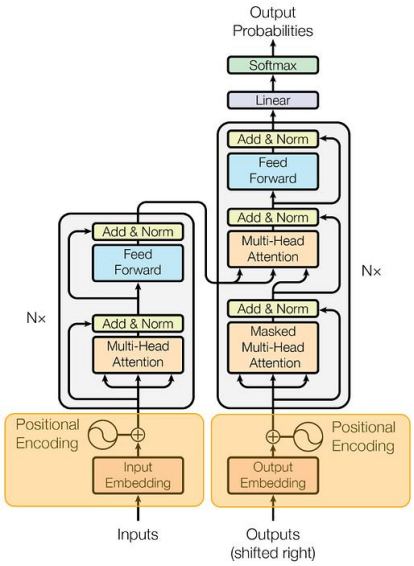
Transformer Architecture

- The Transformer architecture is designed to handle sequence-to-sequence tasks efficiently.
- It consists of an **encoder** and a **decoder**, both of which are built using stacked layers of self-attention and feed-forward neural networks.
- The **encoder** processes the input sequence and transforms it into a continuous representation that the decoder can use.
- The **decoder** generates the output sequence one token at a time, attending to both the encoder's output and the previously generated tokens.



Positional Encoding

- Transformer algorithm does not have any recurrent structure.
  - To address this, positional encoding is added to the input embeddings.
- Positional encoding is a vector that encodes the position of each token in the sequence, allowing the model to capture the order of tokens.
- Positional encodings are added to the input embeddings before they are fed into the encoder or decoder.
- These encodings are designed to have unique values for each position, enabling the model to distinguish between different positions in the sequence.



Formulation for Positional Encoding

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

Training and Inference

Specifications:

- **Model Size**: Original Transformer has 6 encoder and 6 decoder layers (~65M parameters).
- **Context Length**: Limited to 512 tokens due to architecture and positional encoding.
- **Embedding Dimension**: 512 for input embeddings and each layer's output.
- **Model Dimension**: Hidden state size is 512.
- **Attention Heads**: 8 heads, each with dimension 64 (512 ÷ 64).

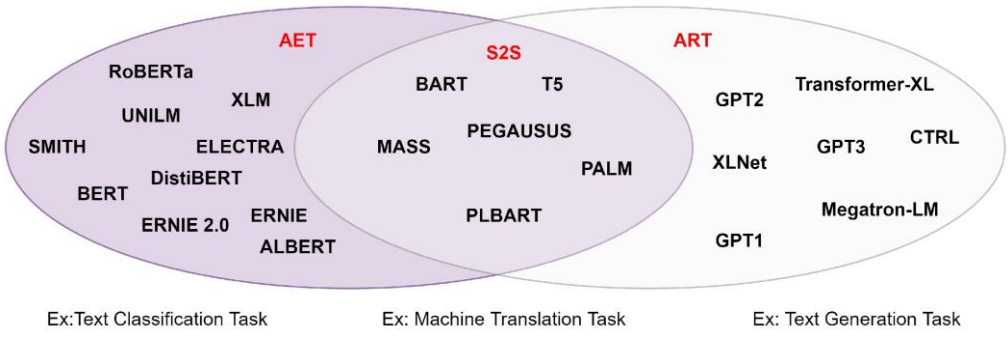
Training:

- The model is trained using teacher forcing, where the ground truth output sequence is fed into the decoder. The loss function is typically the cross-entropy loss between the predicted tokens and the ground truth tokens.

Inference:

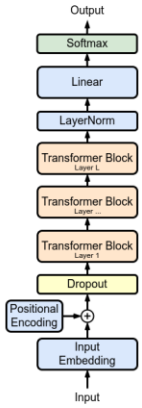
- The model generates the output sequence one token at a time. The previously generated tokens are fed back into the decoder to generate the next token, until the end of the sequence is reached.

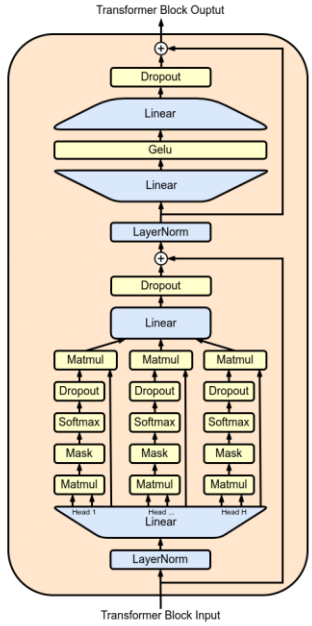
Transformer-based models



Generative pre-trained transformers (GPT)

- GPT (Generative Pre-trained Transformer) is a large-scale transformer-based language model introduced by OpenAI in 2019.
- It achieved state-of-the-art results on several language generation tasks, including text completion and story generation.
- GPT used a pre-training strategy called unsupervised language modeling, where the model is trained to predict the next word in a sequence given the previous words.



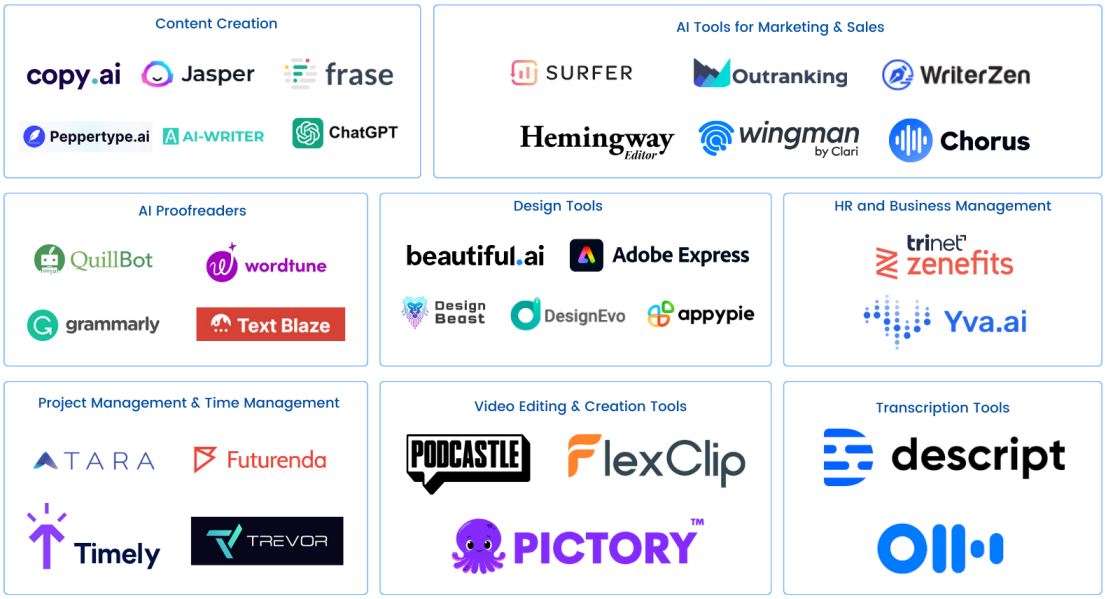


29

GPT versions					
	Uses	Architecture	Parameter count	Training data	Release date
GPT-1	General	12-level, 12-headed Transformer decoder (no encoder), followed by linear-softmax.	117 million	BookCorpus:[3] 4.5 GB of text, from 7000 unpublished books of various genres.	June 11, 2018[4]
GPT-2	General	GPT-1, but with modified normalization	1.5 billion	WebText: 40 GB of text, 8 million documents, from 45 million webpages upvoted on Reddit.	February 14, 2019
GPT-3	General	GPT-2, but with modification to allow larger scaling	175 billion	570 GB plaintext, 0.4 trillion tokens. Mostly CommonCrawl, WebText, English Wikipedia, and two books corpora (Books1 and Books2).	June 11, 2020[5]
InstructGPT	Conversation	GPT-3 fine-tuned to follow instructions using human feedback[6]	175 billion[7]	?	March 4, 2022
ProtGPT2	Protein sequences[8]	Like GPT-2 large (36 layers)	738 million	Protein sequences from UniRef50 (44.88 million total, after using 10% for validation)	July 27, 2022
BioGPT	Biomedical content[9][10]	Like GPT-2 medium (24 layers, 16 heads)	347 million	Non-empty items from PubMed (1.5 million total)	September 24, 2022
ChatGPT	Dialogue	Uses GPT-3.5, and is fine-tuned (an approach to transfer learning)[11] with both supervised learning and RLHF[12]	?	?	November 30, 2022
GPT-4	General	Also trained with both text prediction and RLHF; accepts both text and images as input. Further details are not public.[13]	?	?	March 14, 2023

30

AI in 2026



Large Language Models

- Trained on vast text data to process and generate human language.
- Based on deep learning (e.g., transformers like GPT, BERT, LLaMA).
- Key Capabilities:
  - Text generation (e.g., chatbots, content creation)
  - Summarisation & translation
  - Question answering & information retrieval
  - Text classification (e.g., sentiment analysis)
  - Code generation & automation

LLaMA

Claude

Gemini

ChatGPT